AI食代 SmartFridge ・ 資安詳解

零基礎也能懂的資安說明｜每一項都標明所在檔案、行號與對應程式碼｜可作為上台前的速查與講稿

整體來說，這個專案的資安幾乎集中在兩個檔案：**serverless-handler.ts**（所有後端邏輯：登入、密碼、驗證碼、排程）與 **supabase-rls.sql**（資料庫的逐列權限設定）。以下五大機制，從「它是什麼、為什麼重要、程式在哪、怎麼講」四個角度說明。

1 密碼雜湊：資料庫絕不存明碼 Password Hashing

是什麼：使用者的密碼不會原樣存進資料庫，而是先經過「雜湊」——一種**單向**加密：只能往前算、算不回原文。資料庫裡看到的是一串亂碼。

為什麼重要：就算整個資料庫外洩，攻擊者也拿不到原始密碼。這是帳號安全的第一道、也是最基本的防線。

> **檔案與行號　serverless-handler.ts：hashPassword（第 2721 行）、verifyPassword（第 2728 行）**

```
function hashPassword(password) {
  const salt = randomBytes(16)...      // 每人一撮隨機「鹽」
  const iterations = 260000;           // 反覆計算 26 萬次
  const hash = pbkdf2Sync(password, salt, iterations, 32, 'sha256')...
  return `pbkdf2:sha256:${iterations}$${salt}$${hash}`;
}
```

三個關鍵字（講得出來就贏）：

- **鹽 salt**：加一撮隨機值，讓相同密碼也算出不同結果，破解者無法用預先算好的「彩虹表」查表。
- **迭代 26 萬次**：故意把計算變慢，讓暴力破解一個一個試的成本高到不划算。
- **timingSafeEqual**（verifyPassword 內）：比對時連「花多久」都一樣，避免從反應時間反推密碼。

2 登入狀態：有簽章的 Cookie Signed Session Cookie

是什麼：登入成功後，後端發一張「門票」（Cookie）記著你的使用者編號，並蓋上一個**只有伺服器知道的密鑰**算出的簽章（HMAC）。每次請求都會驗這個簽章。

為什麼重要：別人若想偽造門票、把編號改成別人的，簽章對不上就會被擋，無法冒充身分。

> **檔案與行號　serverless-handler.ts：setSession（第 2682 行）、requireSession（第 2694 行）、sign 用 HMAC-SHA256（第 2705 行）**

```
Set-Cookie: sid=<payload>.<簽章>; HttpOnly; Secure; SameSite=Lax; Max-Age=...

function sign(value) {                 // 用只有後端知道的密鑰簽名
  return createHmac('sha256', 密鑰).update(value).digest('base64url');
}
```

那行 Cookie 的四個字＝四道防線：

- **HttpOnly**：網頁上的 JavaScript 偷不到這個 Cookie（防 XSS 竊取）。

- **Secure**：只在加密的 https 連線傳送。
- **SameSite=Lax**：別的網站不能借用你的登入狀態發請求（防 CSRF）。
- **Max-Age / exp**：門票會過期，過期就要重新登入。

3 忘記密碼與 Gmail 驗證碼：交給 Supabase Auth Password Reset & OTP

是什麼：這些敏感流程不自己刻，而是呼叫 Supabase 官方 Auth，安全性由專業服務保證。

忘記密碼：寄出一條帶「一次性 token」的重設連結；驗證時檢查 token 是否有效、**是否過期**、以及**信箱是否相符**，三關都過才准改密碼。

Gmail 驗證碼（OTP）：免記密碼，寄一組一次性驗證碼；輸錯或過期自動擋，且只發給已註冊帳號。

> **檔案與行號　serverless-handler.ts：requestPasswordReset（第 256 行）、confirmPasswordReset（第 273 行）、requestGmailLogin（第 184 行）、confirmGmailLogin（第 195 行）**

```
// 忘記密碼：驗證 token + 是否過期 + 信箱相符
const authResult = await verifySupabaseOtp({ token_hash: token, type:'recovery' });
if (authUser.email !== email) throw '重設連結與 Email 不相符';
if (newPassword.length < 8)  throw '新密碼至少 8 碼';

// Gmail 驗證碼：只發給已註冊者
requestSupabaseEmailOtp(email)   // should_create_user: false
```

4 RLS 逐列權限：資料庫自己把關 Row-Level Security

是什麼：RLS 讓**資料庫對每一列資料都檢查「你是不是擁有者」**。這是整套資安裡最關鍵的一層。

為什麼重要：就算後端程式哪天寫錯、少加一個過濾條件，資料庫層仍會擋下——你永遠讀不到、也改不了別人的食材與資料。多一層獨立防護。

> **檔案與行號　supabase-rls.sql：enable row level security（第 20 行起）、create policy ... using auth.uid()（第 106 行起）**

```
alter table public.ingredients enable row level security;  -- 開啟逐列檢查

create policy "ingredients_owner_all" on public.ingredients
  for all to authenticated
  using ( ... u.auth_user_id = (select auth.uid()) );  -- 只有擁有者可讀寫
```

白話：auth.uid() 代表「目前登入的人」；policy 的意思就是「這一列的擁有者必須等於你，才准動」。

5 金鑰只在後端 + 排程有密碼 Secrets & Cron Protection

是什麼：能繞過 RLS 做系統維運的高權限金鑰 service_role，只放在後端環境變數，**不進前端、不進 Git**。前端只拿到權限很小的 anon key。

排程保護：每天自動執行的即期提醒排程，用一組 CRON_SECRET 保護，沒帶對密碼就回 401，避免外人亂觸發。

檔案與行號　環境變數見 CURRENT_RUNTIME.md（SUPABASE_SERVICE_ROLE_KEY、GEMINI_API_KEY...）；ensureCronRequest 見 serverless-handler.ts 第 1179 行

```
function ensureCronRequest(req) {
  const secret = process.env.CRON_SECRET || '';
  if (header !== `Bearer ${secret}`) throw '未授權的排程請求';  // 回 401
}
```

資安機制對照表

機制	保護什麼	檔案 / 位置
密碼雜湊 (PBKDF2)	帳號密碼	serverless-handler.ts : 2721
簽章 Cookie (HMAC)	登入狀態	serverless-handler.ts : 2682
忘記密碼 / OTP	身分驗證	serverless-handler.ts : 195 / 273
RLS 逐列權限	使用者資料	supabase-rls.sql : 20 / 106
金鑰只在後端 / Cron	系統與排程	CURRENT_RUNTIME.md ・ handler:1179

一句話總結：密碼雜湊保護帳號、簽章 Cookie 保護登入狀態、RLS 保護資料、金鑰只在後端保護系統。整體採**最小信任原則**——前端永遠拿不到金鑰，也動不了別人的資料。